Nome: João Geiger Piza

Resumo explicando o que é Data Augmentation:

Data Augmentation é uma técnica usada para aumentar artificialmente a quantidade e a fidelidade dos dados de treinamento, criando novas versões a partir de exemplos já existentes. Isso é feito aplicando pequenas alterações nas amostras originais, como rotações, cortes ou mudanças de brilho, sem mudar o conteúdo principal. Assim, o modelo passa a ver mais variações do mesmo dado.

Importância do Data Augmentation em Deep Learning:

Em Deep Learning, o Data Augmentation é importante porque ajuda a reduzir o overfitting, ou seja, quando o modelo aprende demais os dados de treino e funciona mal em dados novos. Com mais variações de entrada, a rede neural consegue generalizar melhor e ficar mais robusta. Além disso, essa técnica é muito útil quando o conjunto de dados é pequeno ou difícil de ampliar.

Exemplos de técnicas de aumento de dados:

- Rotação
- Espelhamento
- Recorte
- Redimensionamento
- Ajuste de brilho e contraste
- Adição de ruído

Descrição dos experimentos realizados com PyTorch:

Os experimentos realizados, de forma breve, envolvem o treinamento de uma CNN a partir de dados que passaram por diferentes processos de aumento de dados (exceto pelo original), e a análise comparativa destes treinamentos.

Respostas e reflexões sobre os exercícios práticos:

1) O uso de rotação aleatória foi incluído na etapa de transformações, obtendo a seguinte nova transforms.Compose:

```
trans = transforms.Compose([transforms.ToTensor(),
transforms.RandomRotation(15), transforms.Normalize((0.1307,),
(0.3081,)),])
```

Ou seja, ele primeiro transforma em tensor, depois aplica uma rotação randômica e, por fim, normaliza.

2) Após a modificação da transformação, observou-se (Observar o gráfico) um treinamento consideravelmente mais lento, com convergência da perda durante o treinamento em torno de 0,06, em comparação com aproximadamente 0,01 no modelo original, e convergência da acurácia em torno de 98,5%, ante aproximadamente 99,7% no modelo original. Assim, como esperado, a introdução de variabilidade nos dados torna o processo de treino aparentemente menos eficiente. Entretanto, ao analisar as curvas de perda e acurácia na validação, verifica-se que o modelo com a transformação aprimorada apresenta um comportamento mais favorável do que o original, pois a curva de perda apresenta uma tendência mais acentuadamente decrescente, de caráter hiperbólico e assintótico, enquanto a curva

de acurácia apresenta uma evolução mais crescente, suave e sigmoidal, com menores oscilações.

3) Eu não aplicaria a transformação ao dataset MNIST, pois um número 2 escrito a mão, com o “lacinho”, poderia ser confundido com um 6, já que os dois teriam um topo curvo, que desce e fecha um círculo em sua base. Outros exemplos com similaridades seriam: o 6 e o 9, já que tem topo curvo e, caso um fosse invertido horizontalmente, iriam descer para o mesmo lado, o 7 e o 5, já que os dois têm topos retos e, caso um fosse invertido horizontalmente, desceriam para o mesmo lado.

Comparação entre os pipelines de treinamento (Resposta do 4 e 5):

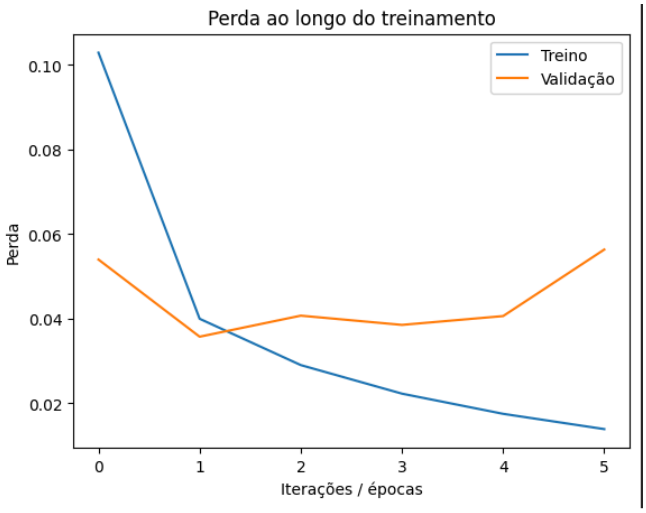
No comparativo direto, o modelo original e o Mixup/CutMix convergem rápido, com perda caindo cedo e acurácia de validação chegando perto de 99%, mas o Mixup/CutMix parece ter uma melhor generalização, porque mantém a validação muito próxima do treino e com um leve ganho final. O pipeline básico com RandomAffine convergiu mais devagar e terminou um pouco abaixo, na faixa de 98,4–98,5%, mas com curvas mais suaves e estáveis. Já o pipeline avançado com TrivialAugmentWide + RandomErasing foi claramente o pior: perda alta, acurácia bem menor, e progresso mais lento; ele não só convergiu menos, como também não chegou perto do desempenho dos outros. Sobre overfitting, o original mostra um sinal leve no fim, o básico quase não mostra, o avançado não chega a esse ponto porque já ficou subajustado, e o Mixup/CutMix reduz bem a distância entre treino e validação.

Na análise geral, uma estratégia prejudicou o desempenho: o pipeline avançado. Ele aumentou demais a dificuldade do treino e acabou reduzindo bastante a acurácia. O melhor resultado foi do Mixup/CutMix, muito próximo do original e com generalização ligeiramente mais limpa. O aumento de dados ajudou quando foi moderado: RandomAffine e Mixup/CutMix preservaram ou melhoraram a robustez sem derrubar muito a acurácia, enquanto a combinação de aumentos fortes degradou o aprendizado. Em termos de estabilidade, o treinamento ficou mais estável no básico e no Mixup/CutMix, e menos estável no avançado, que oscilou mais e aprendeu pior.

Para este problema, eu escolheria Mixup/CutMix. Ele entrega um equilíbrio melhor entre alta acurácia, boa convergência e menor risco de overfitting do que o original puro, sem a perda de desempenho que apareceu no pipeline avançado.

Gráficos de perda e acurácia:

Gráfico do modelo original:



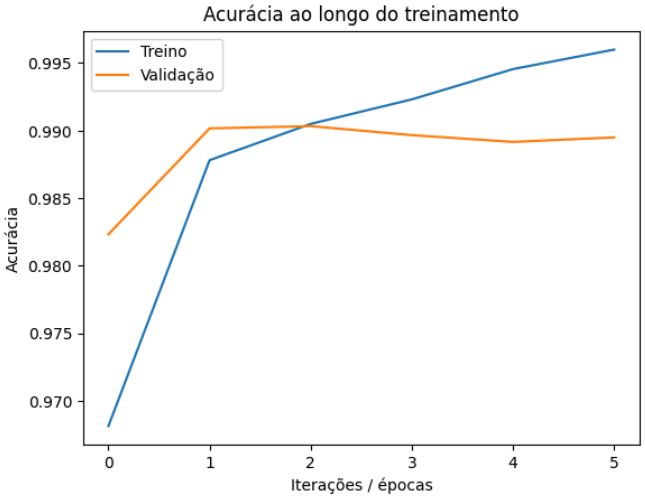
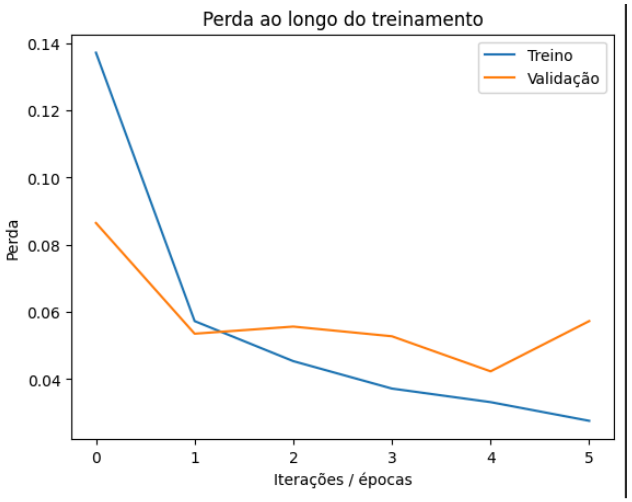


Gráfico do modelo utilizando rotação aleatória:



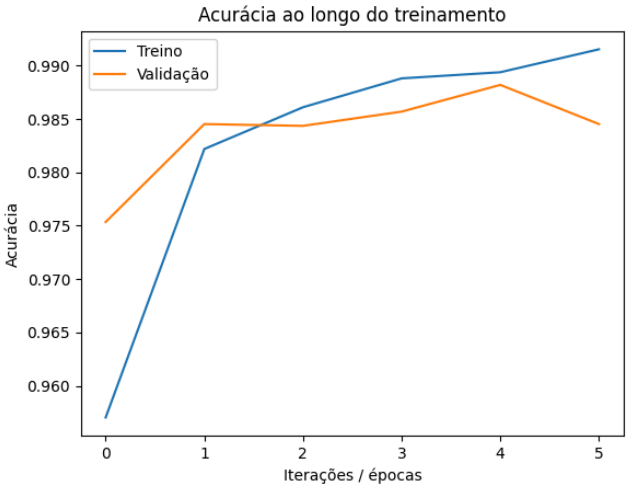
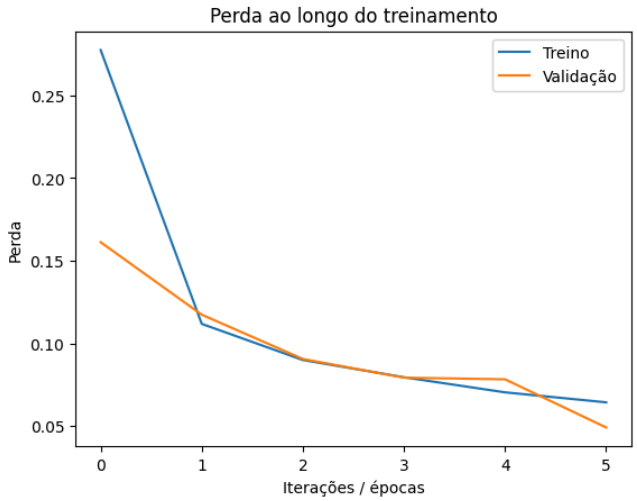


Gráfico após usar a estratégia de transformação do exercício 2:



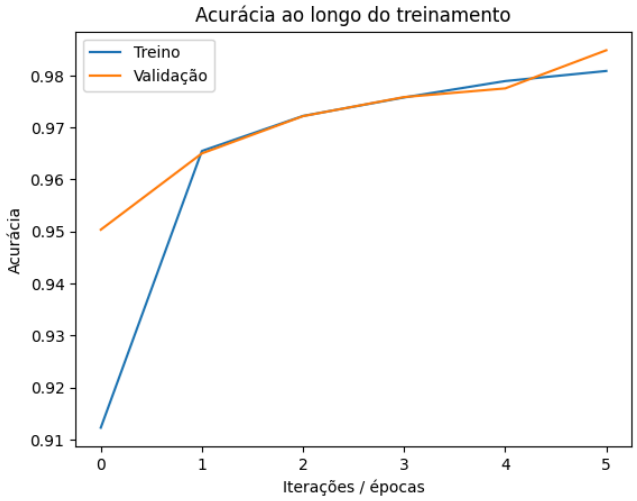
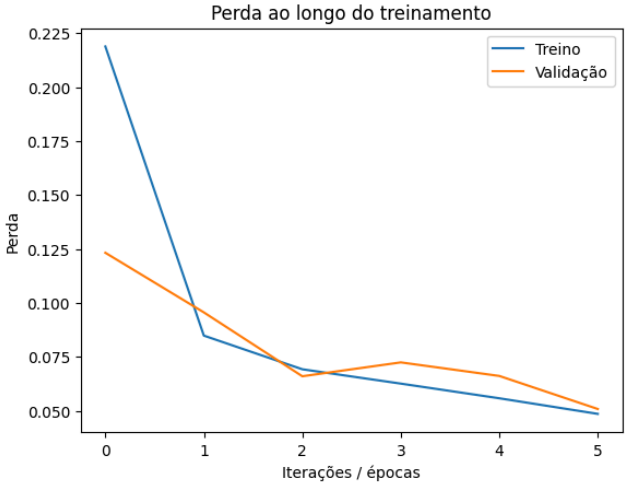


Gráfico do pipeline de treinamento básico:



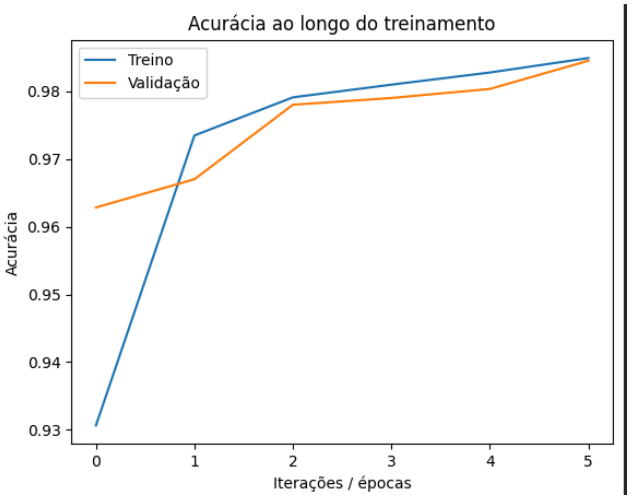
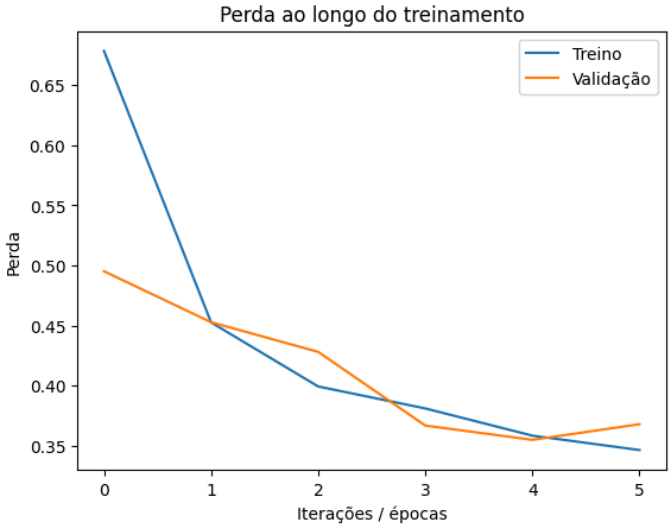


Gráfico do pipeline de treinamento avançado:



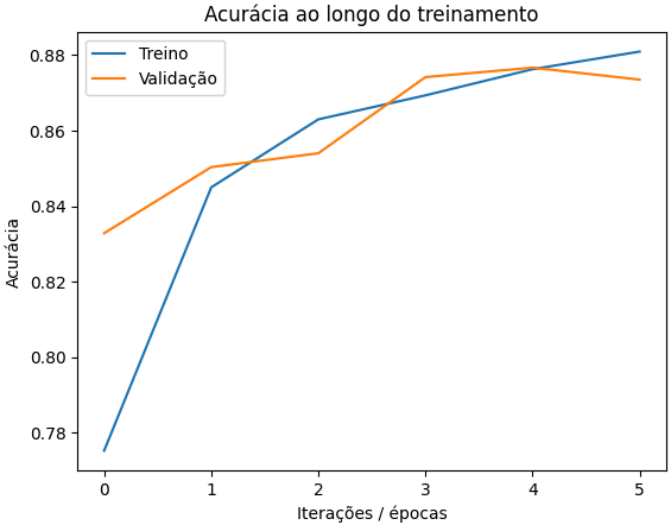
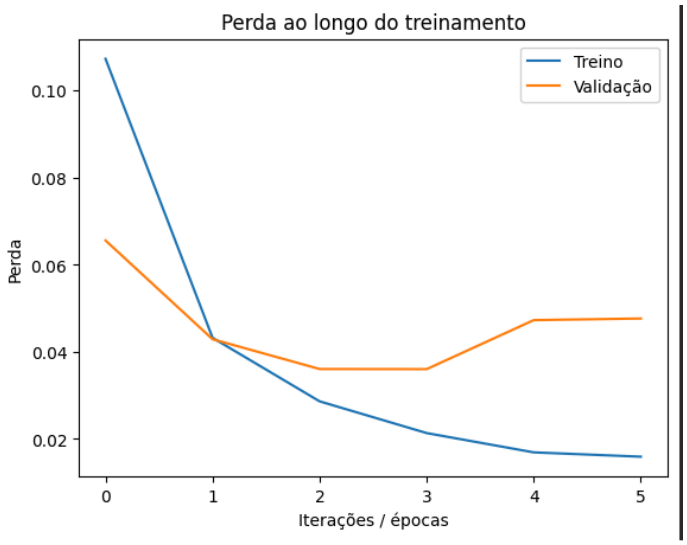
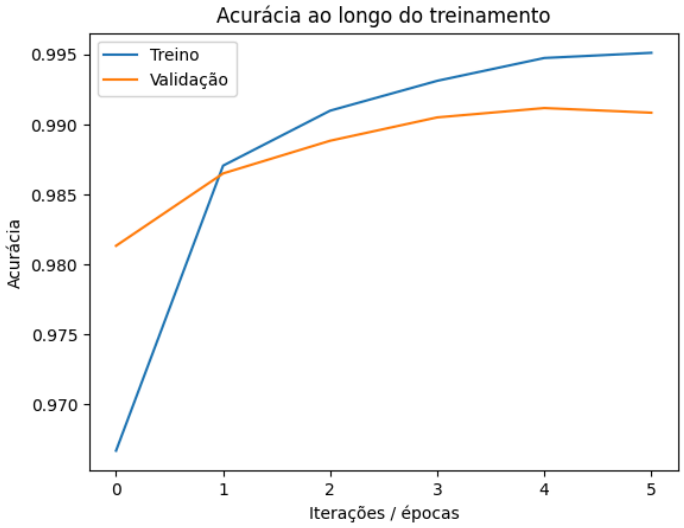


Gráfico do pipeline de treinamento usando MixUp e CutMix, de forma randômica:





Código:

```
import torchvision
from torchvision import transforms

# MNIST dataset
root_path = '/home/storopoli/Downloads' # mude isso no Colab se
necessário

# Pequena transformação para tensores e normalizando o tamanho
trans = transforms.Compose([transforms.ToTensor(),
                            # transforms.RandomRotation(15), # ex 01
                            transforms.Normalize((0.1307,),
(0.3081,))])

trans_aumentado = transforms.Compose([transforms.ToTensor(),
                                      transforms.RandomAffine(0,
scale=(0.9, 1.1),translate=(0.1, 0.1)),
```

```
                                        transforms.RandomPerspective(),
                                        transforms.Normalize((0.1307,),
(0.3081,))])

trans_basico = transforms.Compose([transforms.ToTensor(),
                                   transforms.RandomAffine(15,
translate=(0.1, 0.1)),
                                   transforms.Normalize((0.1307,),
(0.3081,))])

trans_avancado = transforms.Compose([transforms.TrivialAugmentWide(),
                                     transforms.ToTensor(),
                                     transforms.Normalize((0.1307,),
(0.3081,)),
                                     transforms.RandomErasing(),])

# Train/Test Datasets
train_dataset = torchvision.datasets.MNIST(root=root_path, train=True,
transform=trans, download=True)
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_aumentado, download=True)
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_basico, download=True)
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_avancado, download=True)
test_dataset = torchvision.datasets.MNIST(root=root_path, train=False,
transform=trans)

from torch.utils.data import DataLoader
from torch.utils.data import random_split
from torchvision.transforms import v2

batch_size=32
NUM_CLASSES = 100

train_size = int(0.9 * len(train_dataset))
val_size = len(train_dataset) - train_size
train_subset, val_subset = random_split(train_dataset, [train_size,
val_size], generator=torch.Generator().manual_seed(42))
train_loader = DataLoader(dataset=train_subset, batch_size=batch_size,
shuffle=True)
val_loader = DataLoader(dataset=val_subset, batch_size=batch_size,
```

```
shuffle=False)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
shuffle=False)

cutmix = v2.CutMix(num_classes=NUM_CLASSES)
mixup = v2.MixUp(num_classes=NUM_CLASSES)
cutmix_or_mixup = v2.RandomChoice([cutmix, mixup])

for images, labels in train_loader:
    images, labels = cutmix_or_mixup(images, labels)

import torch.nn as nn

class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.fc1 = nn.Sequential(
            nn.Linear(7 * 7 * 64, 1000),
            nn.ReLU())
        self.fc2 = nn.Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.fc1(out)
        out = self.fc2(out)
        return out

# Instancia o Model()
model = ConvNet()

print(model)
```

```python
from torch.optim import Adam

# Hiperparâmetros
loss_fn = nn.CrossEntropyLoss()
learning_rate = 0.001
epochs = 6

# Instânciar o Otimizador Adam
optimizer = Adam(model.parameters(), lr=learning_rate)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
# Treinar o Modelo
total_step = len(train_loader) # quantos batches eu tenho

# Listas vazias
train_loss_list = []
train_acc_list = []
val_loss_list = []
val_acc_list = []

for epoch in range(epochs):
    model.train()
    train_loss = 0
    train_correct = 0
    train_total = 0
    for i, (images, labels) in enumerate(train_loader):
        # Gera a propagação (feed forward)
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)

        # Calcula a função-custo
        loss = loss_fn(outputs, labels)
        train_loss += loss.item()

        # Retro-propagação (Backprop) e a otimização com Adam
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
        # Acurácia
        total = labels.size(0)
        train_total += total
        _, predicted = torch.max(outputs.data, 1)
        correct = (predicted == labels).sum().item()
        train_correct += correct
        if (i + 1) % 100 == 0:
            print(f"Época [{epoch+1}/{epochs}], Step
[{i+1}/{total_step}], Custo: {round(loss.item(), 3)}, Acurácia:
{round((correct / total) * 100, 3)}")
    avg_train_loss = train_loss / len(train_loader)
    train_acc = train_correct / train_total
    train_loss_list.append(avg_train_loss)
    train_acc_list.append(train_acc)
    model.eval()
    val_loss = 0
    val_correct = 0
    val_total = 0
    with torch.no_grad():
       for images, labels in val_loader:
          images = images.to(device)
          labels = labels.to(device)
          outputs = model(images)
          loss = loss_fn(outputs, labels)
          val_loss += loss.item()
          _, predicted = torch.max(outputs.data, 1)
          val_total += labels.size(0)
          val_correct += (predicted == labels).sum().item()
    avg_val_loss = val_loss / len(val_loader)
    val_acc = val_correct / val_total
    val_loss_list.append(avg_val_loss)
    val_acc_list.append(val_acc)

    print(f"Validação - Perda: {round(avg_val_loss, 3)}, Acurácia:
{round(val_acc * 100, 3)}")

import matplotlib.pyplot as plt

plt.figure()
plt.plot(train_loss_list, label="Treino")
plt.plot(val_loss_list, label="Validação")
```

```
plt.xlabel("Iterações / épocas")
plt.ylabel("Perda")
plt.title("Perda ao longo do treinamento")
plt.legend()
plt.show()
plt.figure()
plt.plot(train_acc_list, label="Treino")
plt.plot(val_acc_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Acurácia")
plt.title("Acurácia ao longo do treinamento")
plt.legend()
plt.show()

model.eval() # coloca o modelo em modo de avaliação (sem calcular
gradientes)

with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_loader:
        # Feed-forward com as imagens de teste
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)

        # gera predições usando a função max()
        _, predicted = torch.max(outputs.data, 1)

        # Acumula total e corretas
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    print(f"Acurácia do Modelo em 10k imagens de teste: {round((correct
/ total) * 100, 3)}")
```